Università degli Studi di Verona

CORSO DI LAUREA IN INFORMATICA

Sistemi Operativi

Federico Brutti
federico.brutti@studenti.univr.it

Inserire citazione inerente alla materia

Indice

1 Introduzione ai sistemi operativi **4**
1.1 Algoritmi sequenziali e richiami 4
1.2 Organizzazione di un sistema elaborativo 5
1.3 Attività del sistema operativo 6
1.4 Gestione delle risorse 8

2 Strutture dei sistemi operativi **10**
2.1 Servizi e interfaccia del sistema operativo 10
2.2 Chiamate di sistema 12
2.3 Servizi di sistema, linker e loader 14
2.4 Struttura del sistema operativo 16

3 Processi **20**
3.1 Scheduling e operazioni sui processi 20
3.2 Comunicazione tra processi 26

4 Thread e concorrenza **29**
4.1 Programmazione multicore 29
4.2 Multithreading 29
4.3 Librerie utili 29

5 Scheduling della CPU **30**
5.1 Concetti e algoritmi di scheduling 30
5.2 Scheduling dei thread 30
5.3 Scheduling per sistemi multiprocessore 30
5.4 Scheduling real-time 30
5.5 Valutazione algoritmi 30

6 Sincronizzazione dei processi **31**
6.1 Sezioni critiche 31
6.2 Algoritmo di Peterson 31

6.3 Supporto hardware 31
6.4 Lock/Unlock 31
6.5 Semafori 31
6.6 Monitor 31
6.7 Liveness 31

7 Stallo dei processi **32**
7.1 Stallo e multithread 32
7.2 Caratterizzazioni di stallo 32
7.3 Metodi di gestione e prevenzione 32
7.4 Elusione e rilevamento stallo 32
7.5 Ripristino 32

8 Bash **33**
8.1 Programmazione Bash 33

9 API Posix **34**
9.1 Gestione dei file 34
9.2 Gestione dei diritti 34
9.3 Lettura/scrittura 34
9.4 Gestione dei processi 34
9.5 Segnali 34
9.6 Comunicazione tra processi tramite PIPE 34
9.7 Pthread 34

Capitolo 1

Introduzione ai sistemi operativi

1.1 Algoritmi sequenziali e richiami

Prima di entrare nel vivo del corso è necessario richiamare alcuni concetti e dinamiche visti nel corso di architetture degli elaboratori. Anzitutto, un **algoritmo sequenziale** è un insieme finito di istruzioni elementari, eseguibili in modo deterministico da una macchina astratta. Tale macchina rappresenta un modello teorico di calcolo e possiede le seguenti caratteristiche:

- **Il calcolo è discreto**: L'esecuzione avviene come sequenza di passi elementari, ciascuno dei quali richiede tempo finito.
- **Il comportamento è deterministico**: A parità di stato iniziale e input, l'evoluzione del calcolo è univocamente determinata.
- **La memoria è idealmente illimitata**.
- **Ogni dato manipolato ha dimensione finita**, ma non esiste un limite prefissato alla quantità totale di memoria utilizzabile.

I suddetti algoritmi sono espressi in un linguaggio di programmazione, al quale è associata una macchina astratta capace di eseguire i programmi scritti in tale linguaggio. In particolare, descriviamo tali macchine tramite tre componenti fondamentali:

- **Memoria**: Insieme idealmente infinito di celle per memorizzazione dati, tale che:
  - Ogni cella è identificata da un indirizzo univoco.
  - L'associazione tra celle e indirizzi è biunivoca.
  - Ogni cella può contenere un valore, ed ogni possibile associazione celle-valori memorizzati fornisce una configurazione particolare della memoria.

- **Ambiente**: Associazione tra un insieme finito di identificatori (variabili) e indirizzi di memoria. Consente di collegare i nomi simbolici utilizzati nel programma alle effettive locazioni di memoria.
- **Interprete**: Meccanismo per l'esecuzione delle istruzioni tramite il ciclo fetch-code-execute.

1.2 Organizzazione di un sistema elaborativo

Il **sistema operativo** è lo strato di software che gestisce le risorse hardware di un elaboratore e fornisce servizi a utenti e programmi applicativi. Svolge il ruolo di intermediario tra hardware e utente, con gli obiettivi principali di rendere conveniente l'uso del sistema per l'utente e garantire un utilizzo efficiente delle risorse hardware. Si tratta di una delle quattro componenti dei sistemi di calcolo moderni, insieme all'hardware, gli applicativi e gli utenti.

Figura 1.1: Componenti astratte di un elaboratore moderno

La componente centrale del sistema operativo è il **kernel**, un processo eseguito in modalità privilegiata costantemente in background per fornire meccanismi fondamentali per la gestione degli altri processi e delle risorse. Gestisce direttamente, tra l'altro, CPU, memoria e periferiche I/O.

I programmi che forniscono funzionalità di base non facenti parte del kernel sono detti **programmi di sistema** e sono per esempio la shell o i compilatori, mentre gli altri programmi sono detti **applicativi**. Molti sistemi moderni includono inoltre uno strato software aggiuntivo detto **middleware**, che fornisce servizi di alto livello come database, sistemi grafici o infrastrutture distribuite. All'accensione del calcolatore, l'esecuzione

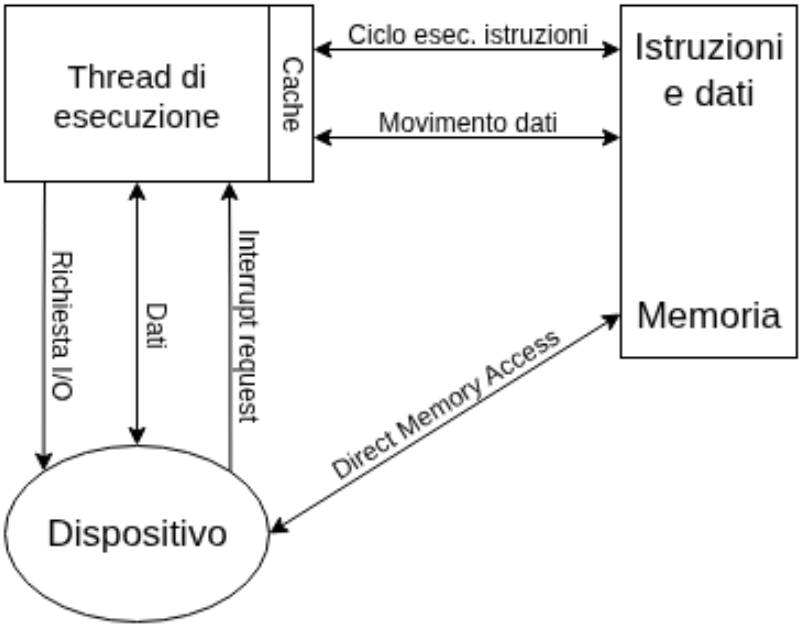


Figura 1.2: Funzionamento di un elaboratore moderno

inizia da un programma **bootloader** residente nel firmware (BIOS o UEFI), il cui compito è inizializzare l'hardware e caricare in memoria un programma di **bootstrap**. Il bootloader provvede quindi a caricare il kernel del sistema operativo in memoria e a trasferirgli il controllo.

Inizializzato, il kernel configura le strutture dati fondamentali, inizializza i dispositivi e avvia i primi processi di sistema. Terminata questa fase, il sistema entra in una modalità di esecuzione guidata dagli eventi. Il controllo del processore passa tra processi e kernel in risposta a interrupt hardware, eccezioni oppure trap, quindi richieste di servizio da parte dei programmi utente.

1.3 Attività del sistema operativo

Fra le principali caratteristiche dei sistemi operativi moderni vi è inoltre la **multiprogrammazione**. Un singolo programma non utilizza costantemente la CPU: durante le operazioni di input/output o in attesa di eventi, il processore rimarrebbe inattivo. Con questo meccanismo si aumenta il suo utilizzo mantenendo in memoria più programmi contemporaneamente.

Il sistema operativo salva temporaneamente in memoria centrale alcuni programmi, in questo caso detti **processi**, e ne sceglie uno per l'esecuzione. Quando il processo scelto deve attendere un evento, la CPU viene assegnata ad un altro processo pronto. Dandole

costantemente nuovi compiti quando sono disponibili si riduce il sul tempo di inattività.

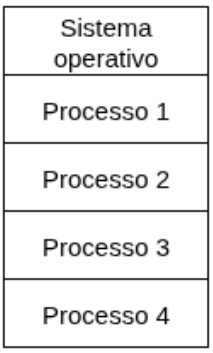


Figura 1.3: Configurazione di memoria per sistemi multiprogrammati

Un'estensione logica della multiprogrammazione è il **multitasking**, detto anche time sharing. Si tratta di un modello nel quale la CPU è assegnata ai processi per intervalli di tempo molto piccoli e limitati, detti **quanti**. Il cambio tra i processi è gestito dallo **scheduler** del processore, ed è effettuato ad una frequenza tale da creare l'impressione di un'esecuzione simultanea, garantendo buoni tempi di risposta.

Poiché le risorse hardware sono condivise, esistono alcuni meccanismi per garantire un'efficiente esecuzione dei programmi, come anche la protezione del sistema operativo. Il primo è dato dalla **memoria virtuale**, la quale consente di eseguire processi non completamente salvati in memoria fisica tramite una sua astrazione. Il secondo riguarda la **modalità duale di funzionamento**, che garantisce protezione tra sistema operativo e programmi utente; in particolare distingue tramite un **bit di modalità** le:

- **Modalità kernel 0**: Per eseguire istruzioni privilegiate.
- **Modalità utente 1**: Per eseguire istruzioni accessibili all'utente.

Siccome l'accesso alle istruzioni critiche è precluso agli utenti, quando un programma necessita di un servizio del sistema operativo, invoca una **system call**. Questa genera una **trap**, vista dal sistema come interruzione, e l'hardware passerà in modalità kernel. Il controllo passa ora a una routine del sistema operativo, individuata tramite un vettore con i valori relativi alle trap. Si esegue ora il servizio richiesto e il controllo ritorna al programma utente in user mode.

Se durante l'esecuzione si verifica una condizione anomala, come per esempio un accesso a memoria non valida o divisione per zero, l'hardware genera un'**eccezione**. In tal caso il controllo passa al sistema operativo, che può terminare il processo, segnalare l'errore ed eseguire eventuali azioni correttive. In alcuni sistemi, se la funzione è presente, possono essere generati dei file testuali detti **memory dump** per analisi successive.

Un ulteriore meccanismo di protezione è rappresentato dal **timer** hardware; un contatore che al raggiungimento dello zero genera un interrupt. Si tratta di una componente essenziale per in primo luogo implementare il time sharing, ma anche per impedire che un processo monopolizzi la CPU, garantendo che il controllo ritorni al sistema operativo entro un certo intervallo di tempo.

1.4 Gestione delle risorse

Per svolgere i propri compiti, un processo richiede alcune risorse, come memoria, file o tempo di CPU. Queste possono essergli assegnate alla sua nascita oppure in esecuzione, per poi essere rese al sistema operativo al termine del processo. In particolare diciamo che i programmi sono entità passive, mentre i processi sono attivi; questi ultimi si dividono in:

- **Single-threaded**: Contengono un solo thread di esecuzione, quindi un unico program counter.
- **Multi-threaded**: Contengono più fili di esecuzione, ciascuno con il proprio program counter e stack.

Tipicamente i sistemi attuano più processi concorrentemente; ciò ha luogo grazie ad un multiplexing della CPU tra **processi e thread**. Per quanto riguarda la loro **gestione**, i sistemi operativi sono responsabili della creazione e cancellazione dei processi utenti e di sistema, schedulazione di processi e thread sulle CPU, sospensione e ripristino dei processi ed infine della fornitura di meccanismi per la loro sincronizzazione e comunicazione.

Per eseguire un programma è necessario che almeno una parte delle sue istruzioni e relativi dati sia scritta nella memoria, in quanto è necessario accedervi per il loro funzionamento. Il programma genera così gli indirizzi logici, che verranno tradotti in indirizzi fisici dal sistema di **gestione della memoria**. In tal merito, il sistema operativo è responsabile di:

- Tenere traccia di quali zone di memoria sono usate e da chi.
- Assegnare e revocare lo spazio di memoria in base alle necessità.
- Decidere quali processi e dati debbano essere caricati in memoria centrale o trasferiti in quella di massa.

Il sistema operativo fornisce inoltre un'interfaccia logica uniforme per la memorizzazione delle informazioni, detta **file**. Si tratta di un'astrazione logica per la memorizzazione indipendente dal dispositivo fisico. Ogni file è organizzato in **directory**, permettendo di ottenere una struttura gerarchica, e possono prendere più forme: **libera**, come per i file testuali, oppure **strutturata**, come per gli mp3. Per quanto riguarda la **gestione dei file**, il sistema operativo è responsabile di:

- Creazione e cancellazione di file e directory.
- Fornitura delle funzioni fondamentali per la gestione di file e directory.
- Associazione dei file ai dispositivi di memoria secondaria.
- Creazione di backup dei file su dispositivi di memorizzazione non volatili.

Poiché i programmi utilizzano i dispositivi di memoria secondaria come dischi rigidi per memorizzare dati e programmi, è di fondamentale importanza che sia gestita correttamente e in modo efficiente. Per la **memoria di massa**, infatti, il sistema operativo **gestisce**:

- Montare e smontare (mounting, unmounting) le unità di memoria.
- Assegnazione dello spazio e gestione dello spazio libero.
- Scheduling del disco.
- Partizionamento.
- Protezione.

Come già detto, più utenti possono usare uno stesso calcolatore ed è quindi necessario gestire l'accesso ai dati mediante apposite regole. Ciò avviene con la **protezione**: meccanismi di controllo dell'accesso alle risorse del PC da parte di processi o utenti. Ci sono varie strategie per attuarla, ma tutte devono fornire le specifiche dei controlli e gli strumenti utilizzati. Un altro concetto molto importante, a causa del costante incremento di attacchi informatici, è la **sicurezza**: la difesa del sistema da attacchi volti al sistema operativo.

Questi due concetti presuppongono che si riesca a distinguere i propri utenti da quelli non autorizzati; generalmente ciò avviene grazie agli identificatori utente, che garantiscono loro unicità. Quando un utente o un gruppo si collega al sistema, infatti, è presente una fase di autenticazione per determinare l'ID corretto e associarlo al processo dell'utente. Se richiesto, è inoltre possibile cambiare l'ID effettivo tramite un comando, ottenendo accesso a comandi privilegiati.

Come ultimo concetto presentiamo la **virtualizzazione**, una tecnica che permette di astrarre l'hardware di un singolo computer, quindi tutte le sue componenti, in diversi ambienti di esecuzione, creando l'illusione di avere più sistemi operativi attivi allo stesso momento su un solo calcolatore, i quali sono capaci di interagire l'uno con l'altro. Otteniamo questa dinamica con le **macchine virtuali**, che consentono agli utenti di cambiare agevolmente il sistema operativo del quale si ha necessità. Un concetto simile è quello dell'**emulazione**, ovvero la simulazione tramite software di un hardware diverso da quello del calcolatore; tecnica usata spesso quando il tipo di CPU della macchina è diverso da quello desiderato.

Capitolo 2

Strutture dei sistemi operativi

2.1 Servizi e interfaccia del sistema operativo

I sistemi operativi garantiscono un ambiente in cui eseguire i programmi e fornire servizi, con lo scopo di facilitare l'interazione con i programmatori di applicazioni. Un primo insieme di servizi riguarda funzionalità utili per gli utenti:

- **Interfaccia con l'utente**: Meccanismo di interazione fra utente e sistema, si divide in **interfaccia grafica utente** GUI, **interfaccia a linea di comando** CLI, e per i dispositivi mobile, **interfaccia touch-screen**.
- **Esecuzione programmi**: La possibilità di caricare un programma in memoria, eseguirlo e terminarlo.
- **Operazioni I/O**: Il sistema operativo fornisce un'interfaccia uniforme per operazioni di input/output su dispositivi quali dischi, tastiera, schermo e rete.
- **Gestione file system**: Lettura, scrittura e ricerca file, creazione ed eliminazione di directory. Fornisce inoltre i metadati.
- **Comunicazioni**: Scambiare informazioni in uno stesso calcolatore e in diversi elaboratori connessi da una rete.
- **Rilevamento errori**: Rilevazione e segnalazione di errori riguardanti la CPU, dispositivi di memoria, I/O o programmi utente. Gestiti mediante arresto del programma o del sistema, oppure con un codice errore.

Un secondo gruppo invece è atto all'efficienza del sistema:

- **Allocazione risorse**: Se più utenti o più processi sono attivi in contemporanea, il sistema operativo provvede alla distribuzione delle risorse necessarie a ciascuno di essi.

- **Accounting**: Mantenere traccia dei programmi usati e quante risorse impiegano.
- **Protezione e sicurezza**: Il sistema operativo evita che i processi interferiscano fra di loro e il sistema stesso e verifica gli utenti tramite richieste di autenticazione.

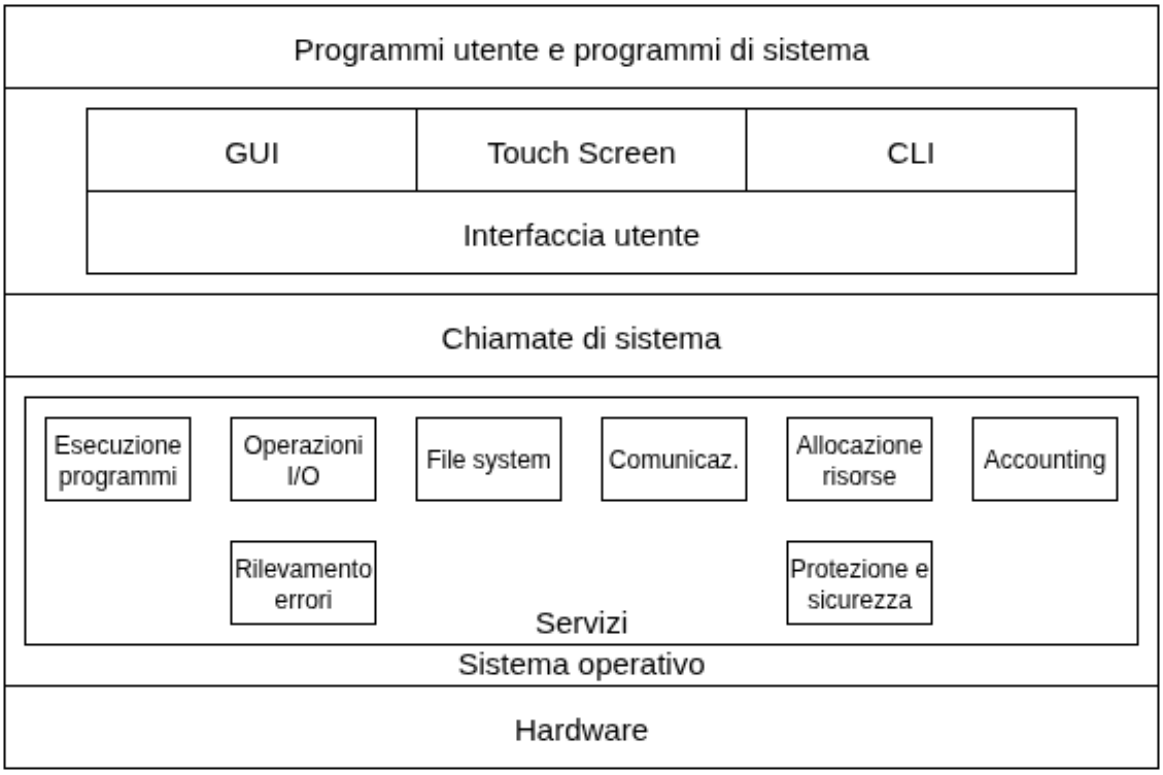


Figura 2.1: Panoramica dei servizi del sistema operativo

Esistono poi due modi fondamentali per gli utenti di comunicare con il sistema operativo: tramite l'**interprete dei comandi** (CLI), implementata da un programma detto **shell**, che permette agli utenti di inserire direttamente le istruzioni da eseguire, e le **interfacce grafiche con l'utente** (GUI), che serve da tramite tra utente e sistema.

Nella maggior parte dei sistemi operativi l'interprete dei comandi è un programma speciale che si attiva all'avvio di un job o al log-in di un utente. La sua funzione principale è raccogliere ed eseguire il comando impartito dall'utente, implementandoli in due modi possibili:

- Se l'interprete contiene il codice per l'esecuzione del comando, come nei casi di gestione file system, salta alla sezione apposita ed invoca le opportune chiamate senza ricorrere a programmi esterni.

- Se non contiene il codice, l'interprete implementa il comando per mezzo di programmi di sistema; non conosce il significato del comando ma si limita ad impiegare il nome per identificare un file da caricare in memoria ed eseguirlo.

Il metodo più utilizzato è il secondo, perché consente la scrittura di nuovi comandi per l'interprete creando nuovi file con un nome appropriato, senza modificare il suo codice.

Per quanto riguarda le interfacce grafiche con l'utente, abbiamo un ambiente nettamente più user-friendly, costituito da una o più finestre con i relativi menu entro cui muoversi con il puntatore del mouse. Usa la metafora della scrivania, il **desktop**, per rendere intuitivi i concetti. Qui si possono trovare le varie **icone**, le quali rappresentano programmi, file, directory (chiamate **cartelle**) e funzioni del sistema. Si tratta del metodo di interazione con l'utente più utilizzato in generale, sebbene i moderni sistemi operativi diano l'opzione per utilizzare entrambi.

2.2 Chiamate di sistema

Le **chiamate di sistema** (system calls) costituiscono un'interfaccia per i servizi resi disponibili dal sistema operativo. Le applicazioni vi possono accedere tramite **Application Programming Interface** (API), fornite dalle librerie digitali, e specificano un insieme di funzioni capaci di comunicare con il sistema operativo. Le API più diffuse sono quella di Windows, POSIX, standard usato per i sistemi UNIX, e quella di Java.

Un altro fattore importante è dato dall'**ambiente di esecuzione al run-time**; una suite di programmi che consente l'esecuzione di applicativi scritti in un determinato linguaggio, collegandolo alle chiamate di sistema tramite un'apposita interfaccia. In base al linguaggio utilizzato, si compone di compilatori, interpreti, librerie digitali, linker e loader.

Ogni chiamata è codificata da un numero specifico, salvato nella **system call table**, a sua volta mantenuta nel kernel e usata dall'**interfaccia alle chiamate di sistema**. Quest'ultima invoca la syscall richiesta, restituendone lo stato. Si possono inoltre presentare in modi diversi, e dipendentemente dal sistema operativo può essere necessario fornire più informazioni come parametri, con uno dei seguenti modi:

- Passaggio del singolo parametro a registro.
- Passaggio di un indirizzo di blocco di memoria a registro.
- Push sulla stack dei parametri.

Inoltre, possiamo raggruppare le chiamate a sistema in sei categorie principali: controllo processi, gestione file, gestione dispositivi, gestione informazioni, comunicazione e protezione. Analizziamole nel dettaglio, fornendo nomi illustrativi delle varie chiamate:

- **Controllo dei processi**: Un programma deve potersi fermare in modo normale o anomalo ($end(), abort()$). In quest'ultimo caso, spesso viene generato un **memory dump**, analizzabile con un debugger. Un processo che esegue un programma può inoltre richiedere di caricare o eseguire ($load(), execute()$) altri programmi tramite una specifica richiesta data da comando utente, clic del mouse o un comando batch. Qui possiamo avere due casi di studio:

  Se al termine del nuovo programma il **controllo torna nel chiamante**, è necessario salvare una sua immagine della memoria; mentre se invece i programmi continuano l'**esecuzione in contemporanea** diciamo che si è creato un nuovo processo in multiprogrammazione $create_process()$.

  Naturalmente è necessario poterne mantenere il controllo e quindi determinare o reimpostare i vari attributi ($get_process_attributes(), set_process_attributes()$), come anche terminare il processo ($terminate_process()$). La terminazione può dipendere dal verificarsi di un certo evento ($wait_event()$) oppure dopo aver aspettato un determinato tempo ($wait_time()$), ma in entrambi i casi bisogna segnalare l'evento ($signal_e vent()$). Capita spesso inoltre che i processi possano condividere dati, rendendo necessario assicurarne l'integrità ed evitare che vengano acceduti o elaborati da più processi allo stesso tempo ($acquire_lock(), release_lock()$).

- **Gestione dei file**: È necessario poter creare e cancellare file ($create(), delete()$), come anche directory. Su questi è possibile eseguire le operazioni di apertura ($open()$), lettura ($read()$), scrittura ($write()$), riposizionamento ($reposition()$) e chiusura ($close()$). Si richiede inoltre di poter determinare i valori dei loro attributi, come nome, tipo o codici di protezione ($get_file_attributes(), set_file_attributes()$). Ciò si ottiene utilizzando delle API apposite.

- **Gestione dei dispositivi**: Per essere eseguito, un processo richiede alcune risorse come spazio in memoria, su disco o accesso a file. Se disponibili, si possono concedere ed il controllo potrà ritornare al processo utente, altrimenti deve attendere fin quando non se ne avranno a sufficienza.

  In caso di utenti multipli, il sistema operativo potrebbe fare una richiesta per l'uso esclusivo di un dispositivo ($request()$), per poi rilasciarlo una volta terminato l'utilizzo ($release()$). Se in ambiente UNIX, è possibile invocare le stesse chiamate a sistema utilizzate per i file, in quanto è uno dei sistemi operativi che li combina in un'unica macrostruttura. Si tratta di una scelta progettuale e non universale.

- **Gestione delle informazioni**: Queste sono chiamate di sistema il cui scopo è trasferire informazioni tra programmi utente e sistema operativo. Alcuni esempi sono l'ottenimento di data e ora oppure la richiesta di effettuare un memory dump ($dump()$). Il sistema operativo contiene informazioni su tutti i suoi processi e sono ottenibili con le chiamate ($get_process_attributes(), set_process_attributes()$).

- **Comunicazione**: Per quanto riguarda lo scambio di informazioni esistono due modelli principali:

  Il **modello a scambio di messaggi** vede processi comunicanti che condividono informazioni tramite una **mailbox** comune. Per prima cosa, si apre un collegamento fra due entità comunicanti il cui nome deve essere noto ed identificato con il **nome di macchina** (*get_hostid*()) e **nome di processo** (*get_processid*()). Questi due identificatori sono poi passati alle chiamate a sistema per aprire e chiudere il collegamento, se è accettato dall'altra entità (*accept_connection*()). Spesso i processi che gestiscono la comunicazione sono demoni specifici; in particolare l'origine dell'informazione è detta **client** ed il ricevente è il **server**. Si servono delle chiamate *read_message*() e *write_message*() per lo scambio e *close_connection*() per terminare la comunicazione.

  Nel **modello a memoria condivisa** invece, si crea crea un'area di memoria condivisa da più processi (*shared_memory_create*(), *shared_memory_attach*()). Questo spazio non è gestito dal sistema operativo ed è necessario che i processi si accordino per non andare in conflitto. Risulta utile perché uno scambio di informazioni su uno stesso elaboratore permette di lavorare alla velocità della memoria e rende il processo più semplice, ma presenta problemi riguardo la protezione e la sincronizzazione dei processi.

- **Protezione**: Il meccanismo di controllo per l'accesso alle risorse del calcolatore. Consente di modificare i permessi di accesso (*set_permission*(), *get_permission*()) a risorse come file e dischi, come anche gestire quelli di utenti specifici, per limitare accessi non autorizzati (*allow_user*(), *deny_user*()).

2.3 Servizi di sistema, linker e loader

I **servizi di sistema** (o system utilities) sono programmi forniti con il sistema operativo che offrono un ambiente per lo sviluppo, l'esecuzione e la gestione dei programmi. In particolare, si classificano nelle seguenti categorie:

- **Gestione dei file**: Compiono operazioni su file e directory come la loro creazione, cancellazione, copia, rinomina e spostamento.

- **Informazioni di stato**: Programmi che forniscono informazioni sullo stato del sistema, come processi attivi, utilizzo della memoria, data e ora o statistiche di prestazione.

- **Modifica dei file**: Programmi per la modifica del contenuto dei file; gli editor di testo.

- **Ambienti di supporto per la programmazione**: Compilatori, assemblatori o interpreti. Spesso forniti direttamente con il sistema operativo oppure installabili separatamente.
- **Caricamento ed esecuzione dei programmi**: Una volta assemblato o compilato il programma, deve essere caricato in memoria per l'esecuzione. Questi programmi sono detti **loader**.
- **Comunicazioni**: Programmi che forniscono meccanismi per creare collegamenti virtuali tra processi, utenti e calcolatori diversi. Consentono lo scambio di informazioni tra processi, utenti o calcolatori attraverso la rete.
- **Servizi in background**: I processi di sistema costantemente in esecuzione, i quali terminano il lavoro all'arresto del sistema. Un esempio è il demone per la gestione delle interfacce di rete.

Oltre a questi servizi, molti sistemi operativi includono ulteriori programmi di utilità per attività comuni, come editor di testo, strumenti di archiviazione e browser web. Generalmente i programmi risiedono su disco in forma di file binario o eseguibile e per essere eseguito dalla CPU è necessario che venga caricato in memoria e associato a un processo. Non è un processo istantaneo, quindi analizziamo fase per fase.

Anzitutto, i **file sorgenti** vengono compilati in **file oggetto rilocabili**, progettati per essere caricati in una qualsiasi posizione della memoria fisica. Il **linker** combinerà ogni file oggetto ed eventuali librerie necessarie in un singolo **binario** eseguibile. Questo è caricato in memoria tramite un **loader** e, allo stesso tempo, avviene la **rilocazione**, che assegna gli indirizzi definitivi alle componenti del programma e riaggiusta il codice e i dati nel programma in base a essi, per garantirne un corretto funzionamento.

Prendiamo ora come esempio l'ambiente UNIX. Per eseguire un programma è necessario dare il relativo comando alla shell. Qui il sistema operativo crea un nuovo processo con la chiamata $fork()$ e richiama il loader con $exec()$, passando il nome del file eseguibile. Viene quindi caricato il programma in memoria usando lo spazio di indirizzamento del processo appena creato. File oggetto ed eseguibili seguono un formato standard che dipende dal sistema operativo di riferimento:

- **UNIX**: Formato Executable and Linkable Format (ELF)
- **Windows**: Portable Executable (PE)
- **MacOS**: Mach-O

Bisogna ricordare che le applicazioni compilate su un sistema operativo non sono generalmente compatibili con altri. Infatti, ogni sistema fornisce un insieme univoco di

chiamate a sistema e una specifica **interfaccia binaria** (ABI), il quale è diversa in altri ambienti. Nonostante ciò, esistono tre modi per rendere disponibili le applicazioni su più sistemi:

- **Uso di un linguaggio interpretato**: I linguaggi come Python o Ruby funzionano grazie ad un interprete; questo legge le righe del codice ed esegue istruzioni equivalenti all'insieme di istruzioni nativo, invocando le chiamate del sistema operativo di riferimento. Dato questo passaggio di traduzione, le prestazioni risultano inferiori rispetto ad applicazioni compilate.
- **Uso di un linguaggio che si serve di macchina virtuale**: Linguaggi come Java utilizzano una macchina virtuale per fornire un ambiente di esecuzione applicazioni indipendentemente dal sistema operativo.
- **Uso di un linguaggio o API standard**: Scrivendo applicazioni che utilizzano interfacce standard come POSIX, è possibile ricompilarle su sistemi diversi con modifiche minime. Sebbene ciò garantisca prestazioni ottimali, il porting può richiedere molto tempo poiché è necessaria una ricompilazione e un adattamento al sistema di destinazione per ogni nuova versione del software.

2.4 Struttura del sistema operativo

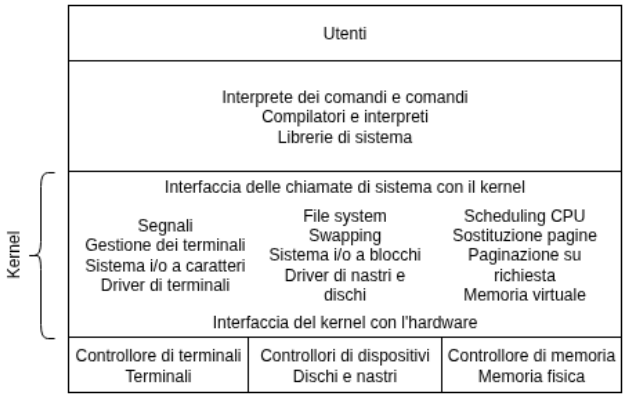
La realizzazione di un sistema operativo vede storicamente protagonista il linguaggio Assembly, ma ad oggi ne vengono normalmente usati di più ad alto livello, come C e C++. Ciò comporta dei vantaggi diretti, come l'avere un codice più compatto, leggibile e più facilmente adattabile ad altre architetture, ma anche svantaggi, come una minore velocità di esecuzione e maggiore utilizzo di spazio di memoria. Tuttavia, grazie alle prestazioni dell'hardware moderno e all'utilizzo di algoritmi e strutture dati migliori, si tratta di problemi ignorabili.

In ogni caso, è necessario prima verificare il corretto funzionamento del sistema operativo, per poi procedere con il miglioramento delle prestazioni, individuando i colli di bottiglia che aumentano i tempi di esecuzione, normalmente le procedure come il gestore degli interrupt, dell'I/O, della memoria e lo scheduler della CPU.

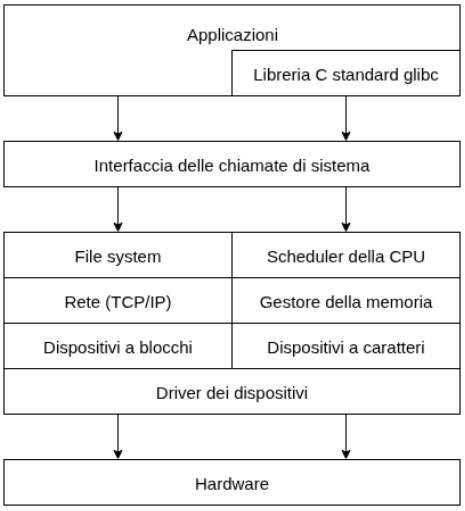
Affinché un sistema operativo possa funzionare ed essere facilmente modificato, sono stati ideati diversi modelli per definirne delle strutture ben precise. Un primo esempio è dato dalla **struttura monolitica**: l'assenza di struttura. Vede tutte le funzionalità del sistema implementate all'interno del kernel, le quali operano nello stesso spazio di indirizzamento.

I vantaggi di questo approccio risiedono nella velocità di comunicazione all'interno del kernel e di un overhead ridotto dell'interfaccia alle chiamate di sistema. Si tratta di una

struttura utilizzata dallo UNIX originale e da Linux, il quale vedremo presenta anche una struttura **modulare**, quindi con delle sottocomponenti che definiscono le parti del sistema.



(a) Struttura di UNIX



(b) Struttura di Linux

La struttura monolitica è caratterizzata da componenti **fortemente accoppiate**, perché tutto è descritto in un solo insieme. Tuttavia è possibile ideare una struttura dai moduli separati, quindi **debolmente accoppiata**, il cui loro insieme compone il kernel. Il vantaggio principale risiede nel mantenimento: le modifiche su una componente non influiscono sulle altre, rendendo notevolmente più semplice il debugging e consentendo una maggiore libertà nella creazione e modifica del codice.

Un'idea che applica questo concetto è l'**approccio stratificato**, il quale vede il sistema operativo diviso in livelli, dove quello più basso rappresenta l'hardware e quello più alto l'interfaccia con l'utente. Ogni strato è composto da strutture dati e un insieme di routine richiamabili dallo strato superiore. La progettazione stessa del sistema operativo sarà guidata da questa struttura; infatti, si passa agli strati superiori solamente quando tutte le funzionalità dei livelli inferiori funzionano correttamente. Sebbene sia decisamente una struttura sicura, in termini di prestazioni risulta scarsa a causa dell'overhead dovuto dall'attraversamento di più strati e per questa ragione non è l'unica idea fondante i sistemi operativi odierni. Tuttavia, si tende ad applicare una parziale stratificazione dove è presente un numero minore di livelli, ognuno con più funzioni. Ciò consente di evitare i problemi connessi alla definizione dell'interazione fra gli strati.

Un altro approccio è quello orientato a **microkernel**: tutti i componenti non essenziali vengono rimossi dal kernel e sono realizzati come programmi di livello utente e di sistema. Ciò che rimane nel kernel di dimensioni ridotte sono i servizi minimi di comunicazione, scheduling e gestione della memoria. Lo scopo principale è quello di fornire

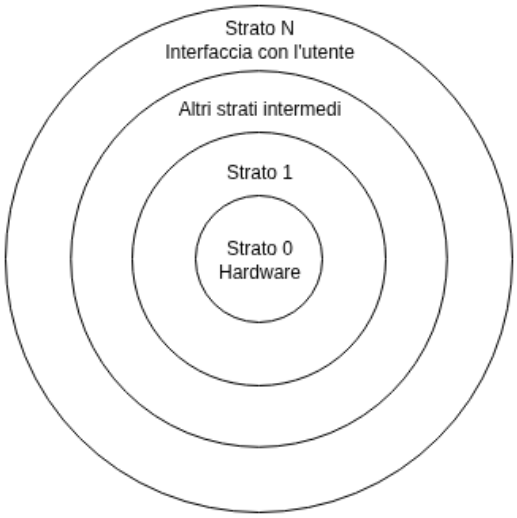


Figura 2.3: Approccio a strati

funzioni di comunicazione tramite scambio di messaggi. Infatti, tra il client e il server, il kernel prende il ruolo dell'intermediario che permette ai due di comunicare.

I principali vantaggi risiedono nella facilità di estensione del sistema operativo e le modifiche minime al kernel quando deve essere aggiornato o modificato. Tuttavia è possibile che presenti cali di prestazioni a causa dell'aumento dell'overhead. Esempi pratici di questo approccio sono sistemi come macOS basati su **Darwin**.

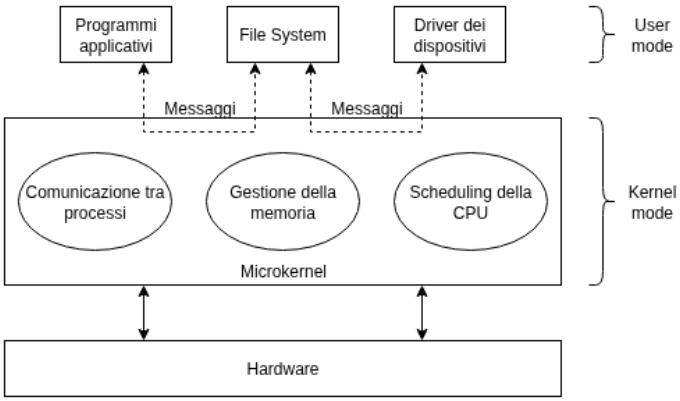


Figura 2.4: Architettura a microkernel

L'approccio attualmente più utilizzato è sicuramente quello basato sui **moduli** del kernel caricati dinamicamente. Il kernel fornisce insiemi di componenti di base integrati da funzionalità aggiunte dinamicamente quando è in esecuzione. Ricorda un sistema stratificato più flessibile con idee di microkernel, perché ogni modulo può chiamarne qualsiasi

altro e il kernel è ridotto alle funzioni minime. Si tratta di un approccio usato per le implementazioni moderne di Linux, Windows e macOS. Per esempio, Linux utilizza **moduli del kernel caricabili** (LKM), i quali sono aggiunti quando necessario per supportare i driver dei dispositivi e i file system. Ciò consente di avere un kernel modulare e dinamico, mantenendo allo stesso tempo le prestazioni di un sistema monolitico.

I sistemi operativi odierni non adattano una sola idea architetturale, bensì ne applicano più alla volta. Per esempio, come menzionato prima, Linux utilizza di base un approccio monolitico, ma è allo stesso tempo modulare poiché consente di caricare componenti al kernel dinamicamente. Anche Windows presenta una struttura simile, applicando tuttavia idee orientate ai microkernel, come per esempio il supporto a sottosistemi separati.

Capitolo 3

Processi

3.1 Scheduling e operazioni sui processi

Si ritiene necessario dare un nome alle attività compiute dalla CPU: in particolare, i sistemi **batch** eseguono **job**, mentre quelli **time-sharing** dei **task**. Chiamiamo queste attività col nome generale di **processo**.

Intuitivamente, un processo è un programma in esecuzione il cui stato è rappresentato da program counter e registri del processore. Si compone di:

- **Sezione di testo**: Contiene il codice eseguibile.
- **Sezione dei dati**: Contiene le variabili globali.
- **Heap**: Memoria allocata dinamicamente durante l'esecuzione del programma.
- **Stack**: Memoria usata temporaneamente nelle chiamate a funzione.

È importante notare che heap e stack sono di dimensione variabile e crescono l'una verso l'altra; è compito del sistema operativo garantire che non si sovrappongano. La prima aumenta o diminuisce di dimensione grazie a funzioni come malloc e free per il linguaggio C, mentre le chiamate a funzione inseriscono un **record** di attivazione contenente parametri, variabili locali e indirizzo di ritorno sulla stack, per poi rimuoverlo quando il controllo del programma ritorna al chiamante.

Noi sappiamo che un programma diventa processo quando il suo eseguibile è caricato in memoria e sebbene due processi possono essere associati ad uno stesso programma, sono da considerare come due istanze diverse, le quali seguono il proprio percorso di esecuzione, detto **thread**. Inoltre, durante la loro vita, i processi possono presentarsi nei seguenti stati:

- **Nuovo**: Il processo è appena creato.

- **In esecuzione**: Le istruzioni del processo stanno venendo eseguite.
- **In attesa**: Il processo attende il verificarsi di un evento.
- **Pronto**: Il processo attende di essere assegnato a un'unità di elaborazione.
- **Terminato**: Il processo ha terminato l'esecuzione.

Questi stati sono comuni a tutti i sistemi operativi, sebbene alcuni introducano ulteriori distinzioni. È bene inoltre notare che sebbene più processi possano essere pronti, i core delle unità di elaborazione possono eseguirne solo uno per volta.

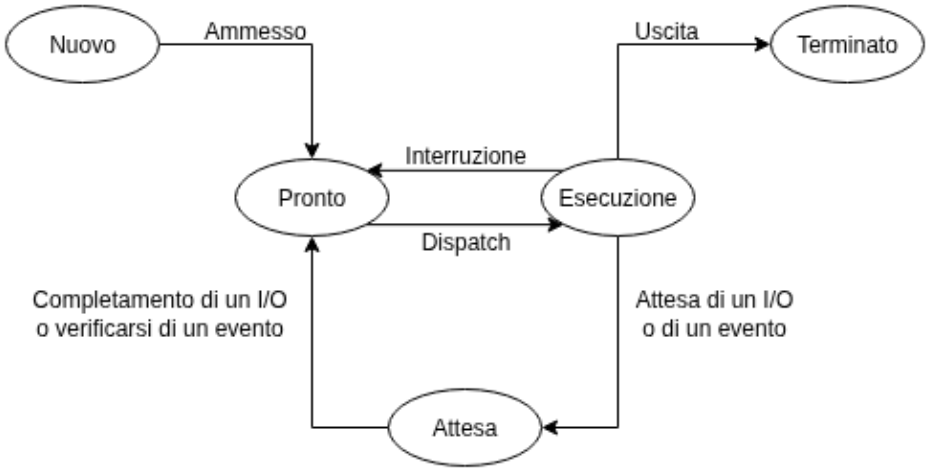


Figura 3.1: Diagramma degli stati di un processo

Nei sistemi operativi i processi si rappresentano con **blocchi di controllo** (Process Control Block, PCB), depositi di informazioni per ogni processo specifico. Salvano dati in merito allo stato corrente, il valore del program counter, lo stato dei registri della CPU e informazioni sul suo scheduling, informazioni sulla gestione della memoria, di accounting e infine sullo stato dell'I/O.

Nei sistemi operativi moderni si è esteso il concetto di processo introducendo la possibilità di avere più percorsi di esecuzione, permettendo ai processi di svolgere più compiti alla volta. In un sistema che supporta i thread, infatti, il PCB è esteso per includere informazioni su di essi.

Riprendiamo ora il concetto di **multiprogrammazione**, il cui obiettivo è avere almeno un processo sempre in esecuzione per massimizzare l'utilizzo della CPU. Il cambio tra i vari processi avviene così frequentemente da consentire l'interazione con ciascun programma durante la loro esecuzione ed è gestita dallo **scheduler**, il quale seleziona il

processo e lo assegna al core della CPU. Le architetture odierne comprendono CPU a più core, ed in particolare chiamiamo **grado di multiprogrammazione** il numero di processi in memoria in un dato istante.

Il bilanciamento tra multiprogrammazione e time sharing richiede di tener conto anche del comportamento complessivo dei processi: definiamo infatti **I/O-bound** i processi che impiegano la maggior parte del tempo nell'esecuzione di operazioni input/output e **CPU-bound** quelli che eseguono nella maggior parte elaborazioni. Ogni processo è inizialmente inserito dallo scheduler in una **coda dei processi pronti**, tipicamente implementata come una struttura dati, in attesa di essere assegnato ad un core. Esiste inoltre una **coda di attesa** dove sono spostati i processi in attesa del completamento di un evento, come la ricezione di un input.

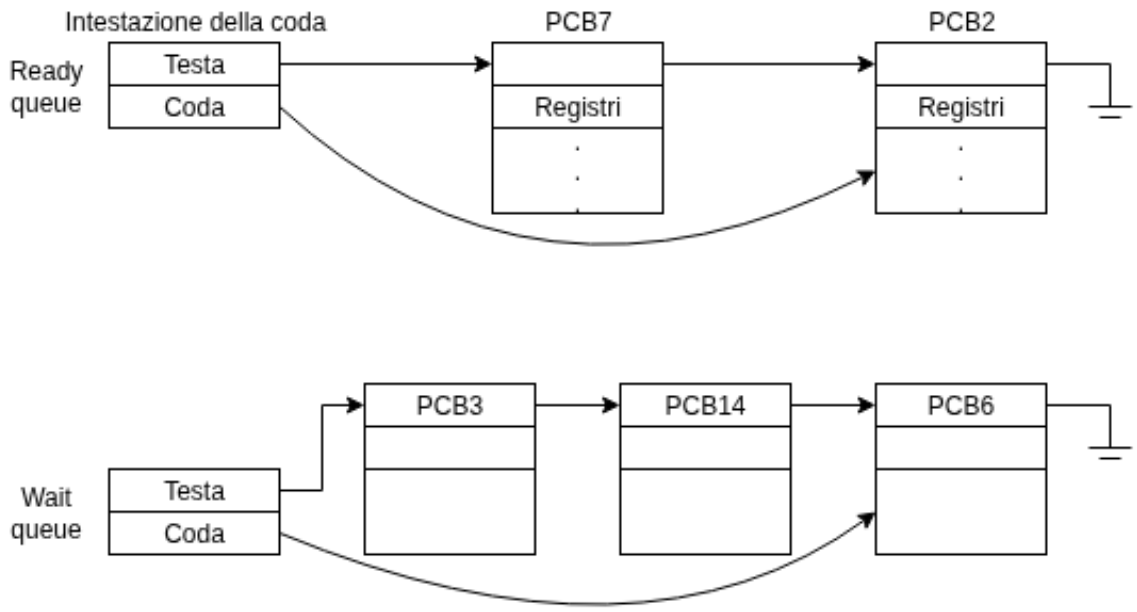


Figura 3.2: Coda dei processi pronti e di attesa

Con l'inserimento nella coda dei processi pronti, si possono verificare i seguenti casi:

- Processo emette richiesta I/O, viene spostato quindi nella coda di I/O.
- Processo crea un processo figlio, attendendo la sua terminazione.
- Processo rimosso forzatamente dalla CPU a causa di interruzione. Viene reinserito nella coda dei processi pronti.

Nei primi due casi, al completamento della richiesta I/O o al termine del processo figlio, il processo originale passa dallo stato di attesa a quello pronto ed è nuovamente inserito

nella coda dei processi pronti. Il ciclo continua fino al termine dell'esecuzione e si rimuoverà quindi da tutte le code, deallocando anche il PCB e le risorse legate al processo.

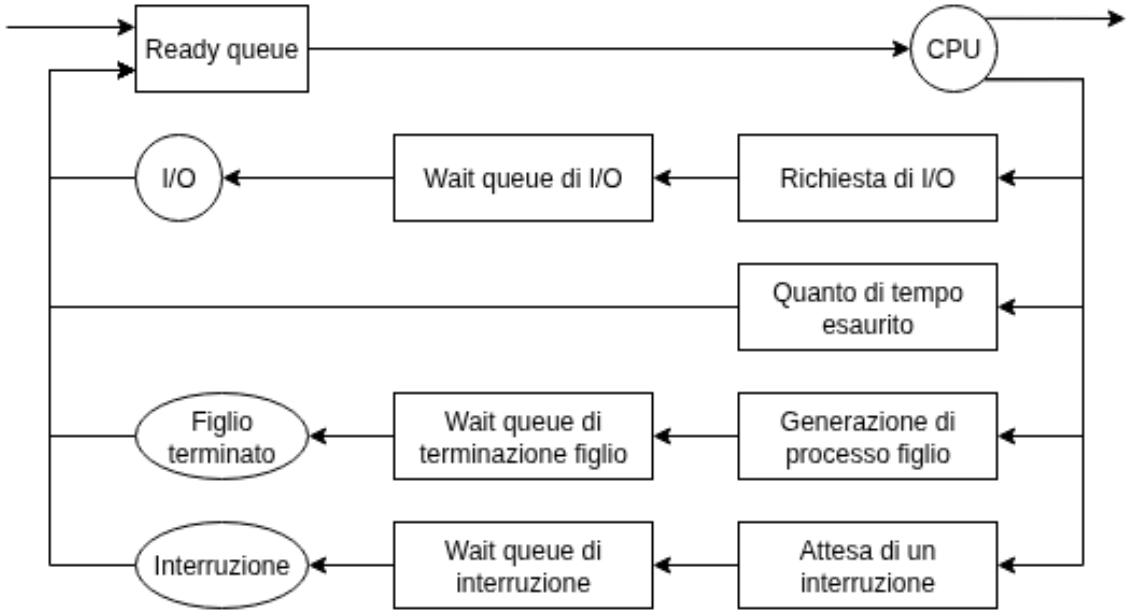


Figura 3.3: Diagramma di accodamento per lo scheduling

Come già detto, i processi si spostano tra le varie code disponibili frequentemente; alcuni sistemi operativi, per ridurre il grado di multiprogrammazione, implementano un algoritmo intermedio nello scheduling per rimuovere forzatamente i processi dalla CPU, detto **avvicendamento dei processi in memoria**.

Viene quindi salvato lo stato al momento dello stop del processo rimosso, per poi riprendere da dove è stato interrotto quando lo si reinserisce in memoria. Inoltre, se è implementato lo **swapping**, i processi si possono spostare dalla memoria centrale al disco per un ulteriore risparmio di spazio. Il salvataggio e ripristino dello stato della CPU si dice invece **cambio di contesto** e il tempo necessario per attuarlo varia in base alla velocità dell'architettura.

Nella maggior parte dei sistemi operativi i processi lavorano concorrentemente. Si ritiene dunque necessario fornire un meccanismo per la loro creazione e terminazione. I processi genitori possono crearne di figli, i quali a loro volta possono crearne altri. Ciò forma un albero di processi. Per poterli distinguere, i sistemi operativi come Linux li identificano tramite un **identificatore di processo** (pid), un valore numerico intero univoco per ogni processo.

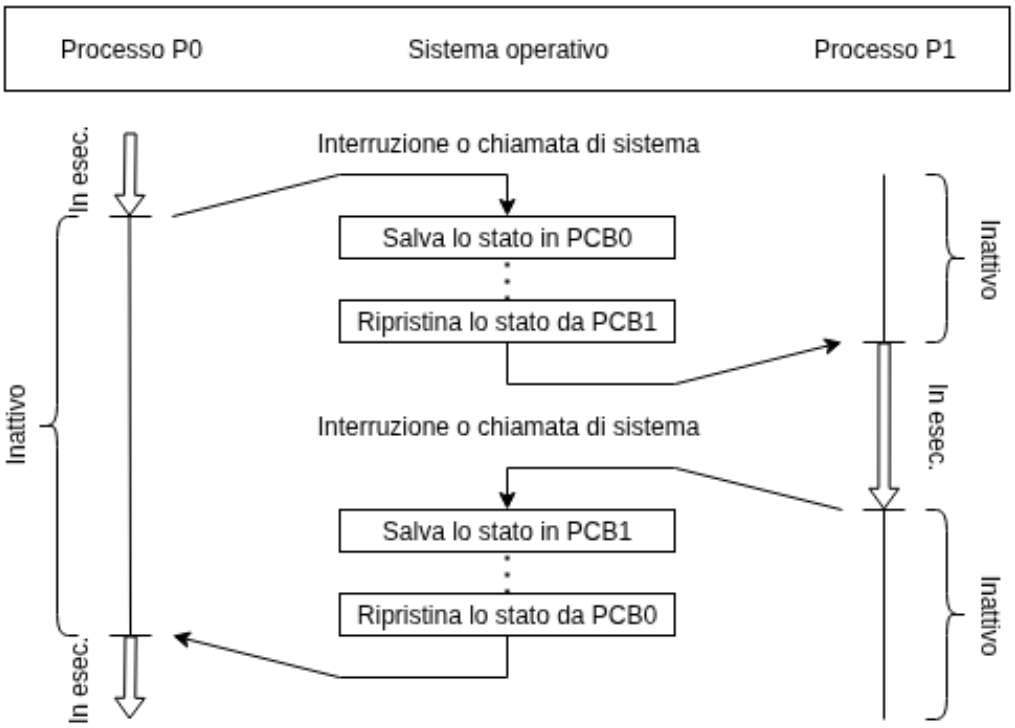


Figura 3.4: Diagramma di cambio di contesto

Generalmente, la creazione di un processo implica l'impiego di determinate risorse: ciò vale sia per genitori che figli. Questi ultimi, in particolare, possono riceverle direttamente dal sistema operativo oppure da un insieme ristretto del genitore. Quest'ultima opzione risulta preferibile in quanto limitando le risorse di un figlio si può evitare che il processo sovraccarichi il sistema usando troppa memoria.

Oltre alle varie risorse fisiche e logiche, il genitore può passare ai figli dei dati di inizializzazione, non dissimilmente da una chiamata a funzione. Possiamo avere le seguenti istanze:

- Il genitore continua l'esecuzione concorrentemente al figlio.
- Il genitore attende la terminazione di alcuni o tutti i processi figli.

Inoltre, ci sono due possibilità per quanto riguarda lo spazio di indirizzi del figlio:

- Il figlio è un duplicato del genitore e ha stessi dati e programma.
- Nel figlio si carica un nuovo programma.

Prendiamo come esempio il workflow di UNIX, dove i nuovi processi sono identificati con un pid, generati con fork() e composti da una copia dello spazio di indirizzi del genitore. Ciò consente una facile comunicazione tra genitore e figlio.

Entrambi i processi continuano la loro esecuzione all'istruzione successiva al fork(), il quale riporta il valore zero nel nuovo processo, ma ritorna il pid del figlio al genitore, per identificarlo. In genere, dopo la fork(), uno dei due processi impiega una exec() per sostituire lo spazio di memoria del processo con un nuovo programma. La chiamata carica in memoria un file binario cancellando l'immagine di memoria del programma che la contiene e avvia la sua esecuzione. Così i due processi possono comunicare e procedere in modi diversi.

Il genitore può, come detto prima, generare più figli o aspettare la loro terminazione invocando wait(). La sua chiamata consente di rimuovere sé stesso dalla coda dei processi pronti fino alla terminazione del figlio. Inoltre, siccome exec() sovrappone lo spazio di indirizzi del processo con un nuovo programma, non può restituire il controllo a meno che non si verifichi un errore. Usando la chiamata execlp(), il figlio sovrappone il proprio spazio di indirizzi con una stringa, per esempio execlp("/bin/ls"), ovvero il comando per ottenere l'elenco dei file in una directory. Quando un processo figlio termina, chiamando implicitamente o esplicitamente exit(), il genitore chiude la fase di attesa data da wait() e termina a sua volta con exit().

Un processo termina quando è stata eseguita la sua ultima istruzione e inoltra la richiesta di terminazione al sistema operativo con exit(). Se è un figlio, riporta un'informazione di stato al genitore se ha invocato wait(). Tutte le risorse del processo saranno ora liberate dal sistema operativo. I processi possono essere terminati anche per altre ragioni, come un'interruzione, chiamabile solo dal padre, il quale deve necessariamente conoscere il pid del figlio per poterlo terminare. Generalmente, un genitore può porre fine alla vita di un figlio per svariati motivi, tra cui:

- Il figlio ha ecceduto nell'uso delle risorse.
- Il compito assegnato al figlio è terminato.
- Il genitore termina la propria esecuzione. In particolare, i processi terminati il cui padre è ancora attivo sono detti **zombie**, mentre se il padre termina senza invocare wait(), i figli saranno adottati da systemd per la chiusura e si dicono **orfani**.

Se un processo termina, così dovranno fare anche gli eventuali figli: questo meccanismo è detto **terminazione a cascata** ed è una procedura avviata dal sistema operativo. Non è tuttavia comune a tutti i sistemi operativi.

La terminazione fornisce uno stato di uscita, dove lo 0 indica una terminazione normale, mentre altri valori segnalano terminazioni anomale. Segue la deallocazione delle risorse utilizzate per il processo.

3.2 Comunicazione tra processi

All'interno dei sistemi operativi definiamo due tipi di processi: **indipendenti** se non influiscono né sono influenzati dagli altri e **cooperanti** quando influenzano o sono influenzati. I motivi per la creazione di un ambiente cooperativo tra i processi sono:

- **Condivisione di informazioni**: Più utenti potrebbero volere accesso alle stesse informazioni.
- **Velocizzazione del calcolo**: Alcune operazioni sono più rapide quando divise in sottoattività eseguibili in parallelo.
- **Modularità**: Suddivisione delle funzioni in processi o thread distinti.

I processi cooperanti richiedono di un meccanismo per la comunicazione; i due principali si dicono a **memoria condivisa**, richiedente una zona di memoria allocata dinamicamente condivisa dai processi, e a **scambio di messaggi**, dove la comunicazione avviene tramite l'omonimo meccanismo.

Il primo paradigma può risultare più veloce rispetto all'altro perché richiede l'intervento del kernel esclusivamente per definire la zona condivisa, mentre lo scambio di messaggi è implementato tramite chiamate di sistema.

In particolare, per i sistemi a memoria condivisa, la zona allocata si trova nello spazio degli indirizzi del processo che la riserva e richiede un accordo tra le entità che devono utilizzarla, poiché la sua gestione è fuori dal dominio del sistema operativo. Fondamentalmente è utilizzato un paradigma di **produttore-consumatore**, il quale vede un primo processo produrre informazioni che verranno consumate dal secondo.

L'esecuzione concorrente di due processi comunicanti avviene infatti grazie a un **buffer** che può essere riempito dal produttore e svuotato dal consumatore. Naturalmente deve esserci una sincronizzazione tra i due per fare in modo che non si consumino informazioni prima di crearle. Ci sono inoltre due tipi di buffer utilizzabili:

- **Illimitato**: Consente al produttore di creare sempre nuove informazioni.
- **Limitato**: Pone una dimensione al buffer condiviso; il consumatore dovrà attendere se è vuoto e il produttore dovrà attendere se è pieno.

In una comunicazione basata sullo scambio di messaggi è invece fondamentale fornire strumenti per la comunicazione e un meccanismo di sincronizzazione dei processi senza condividere lo spazio di indirizzi. Si prevede sempre l'utilizzo di almeno due operazioni primitive: send() per l'invio dei messaggi e receive() per la loro ricezione.

I messaggi possono presentarsi con lunghezza fissa o variabile e sono inviati grazie a un apposito **canale di comunicazione**, realizzabile in più modi secondo la tipologia scelta

tra **diretta/indiretta**, **sincrona/asincrona** oppure con gestione **automatica/esplicita** del buffer. Per comunicare è inoltre necessario che i processi possano fare riferimento ad altri, quindi conoscere i loro identificativi.

In una **comunicazione diretta** ogni processo che intende comunicare deve nominare esplicitamente il ricevente o trasmittente e definisce le primitive come segue:

- send(P, message): Invia message al processo P.
- receive(Q, message): Riceve message dal processo Q.

In uno schema tale, devono valere però alcune caratteristiche: in primo luogo, tra ogni coppia di processi comunicanti si deve instaurare automaticamente un canale ed entrambi devono conoscere solamente la reciproca identità. Il canale sarà uno solo ed è associato esclusivamente a una coppia di processi.

Una **variante** di questo modello presenta un'asimmetria nell'indirizzamento; solamente il processo che invia il messaggio nominerà il ricevente, modificando le primitive come segue:

- send(P, message): Invia message al processo P.
- receive(id, message): Riceve un messaggio da qualsiasi processo, identificato con id.

Entrambi questi schemi presentano una scarsissima modularità, rendendoli poco ideali se è necessario effettuare modifiche di qualche tipo. Un'idea più agevole è invece data dalla **comunicazione indiretta**, la quale impiega delle **porte** o **mailbox** univocamente identificate per la ricezione dei messaggi. In un modello tale, i processi possono comunicare esclusivamente se condividono una cassetta e modifica le primitive in questo modo:

- send(A, message): Invia message alla mailbox A.
- receive(A, message): Riceve message dalla mailbox A.

Ne consegue che un canale di comunicazione può esistere solamente se due o più processi condividono una mailbox, ma ciò rende i canali non più esclusivi a soli due processi. Tra ogni coppia possono infatti essercene diversi, ognuno dei quali corrispondente a una mailbox.

Dando l'accesso a più di due processi ad una sola mailbox si introduce il problema della decisione del destinatario del messaggio. Non c'è una dinamica deterministica per la scelta del processo ricevente, quindi si sceglie di decidere arbitrariamente con una **cablatura**[1] oppure di scrivere un algoritmo per la selezione del destinatario, come quello

[1] Le cablature, in inglese **hard-coding**, sono generalmente considerate cattiva prassi.

di **round robin**, che assegna i messaggi ai processi ciclicamente, e comunicare l'identità del ricevente al trasmittente.

Una mailbox può appartenere sia a processi che al sistema operativo; nel primo caso, si trova nello spazio di indirizzi del processo e occorre distinguere tra proprietario e utente. Quando termina, la relativa casella scompare ed è inviato un messaggio a tutti gli altri processi in essa comunicanti.

Quando la mailbox è invece posseduta dal sistema operativo, presenta una vita autonoma ed è quindi indipendente da altri processi. Il sistema offre inoltre un meccanismo per la creazione e rimozione di mailbox, come anche la ricezione e invio di messaggi. Quando un processo crea una mailbox con il sistema operativo, ne è il proprietario predefinito, ma è possibile modificarne la proprietà con apposite chiamate di sistema.

Come già menzionato, lo scambio di messaggi può avvenire tramite due modalità: **sincrona**[2], dove il mittente si blocca finché il messaggio non è inviato ed il destinatario si sospende finché non è disponibile un messaggio, e **asincrona**, che vede le due entità continuare la propria esecuzione dopo invio o ricezione del messaggio. È inoltre possibile avere più combinazioni di primitive in quanto non sono mutualmente esclusive. Infine, quando la comunicazione è diretta o indiretta, i messaggi scambiati risiedono in **code temporanee**, delle quali esistono tre tipi:

- **Capacità zero**: Il canale di comunicazione non può ospitare messaggi; il trasmittente si blocca finché il ricevente non è pronto a ricevere il messaggio.
- **Capacità limitata**: La coda può ospitare un numero n di messaggi. Quando questa è piena, il trasmittente blocca la sua esecuzione e attende l'invio del messaggio, altrimenti invia il contenuto senza fermarsi.
- **Capacità illimitata**: La coda non ha limiti di spazio; il trasmittente non si ferma mai.

[2]Se mittente e destinatario hanno comportamento sincrono, il loro incontro è detto **rendez-vous**.

Capitolo 4

Thread e concorrenza

4.1 Programmazione multicore

4.2 Multithreading

4.3 Librerie utili

Capitolo 5

Scheduling della CPU

5.1 Concetti e algoritmi di scheduling

5.2 Scheduling dei thread

5.3 Scheduling per sistemi multiprocessore

5.4 Scheduling real-time

5.5 Valutazione algoritmi

Capitolo 6

Sincronizzazione dei processi

6.1 Sezioni critiche

6.2 Algoritmo di Peterson

6.3 Supporto hardware

6.4 Lock/Unlock

6.5 Semafori

6.6 Monitor

6.7 Liveness

Capitolo 7

Stallo dei processi

7.1 Stallo e multithread

7.2 Caratterizzazioni di stallo

7.3 Metodi di gestione e prevenzione

7.4 Elusione e rilevamento stallo

7.5 Ripristino

Capitolo 8

Bash

8.1 Programmazione Bash

Capitolo 9

API Posix

9.1 Gestione dei file

9.2 Gestione dei diritti

9.3 Lettura/scrittura

9.4 Gestione dei processi

9.5 Segnali

9.6 Comunicazione tra processi tramite PIPE

9.7 Pthread